ResearchScope

Intelligent Research Analyzer: From Classical NLP to Agentic AI

End-Semester Project Report

Team Members:

Kushal Sarkar
Chinmay Soni
Lakshya Bapna

Course: Generative AI (Section A)

Submission: End-Semester Evaluation

Date: April 2026

Live Application: researchscopegenai.streamlit.app
GitHub Repository: github.com/Kushal425/GENai_SecA_P1

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Problem Statement . . . 2
2.2 Objectives of Milestone 2 . . . 2
2.3 Constraints . . . 3

3 Milestone 1: Classical NLP Pipeline (Summary) **3**
3.1 Pipeline Overview . . . 3
3.2 M1 Evaluation Results . . . 4
3.3 Limitations that Motivated Milestone 2 . . . 4

4 Milestone 2: Agentic AI Research Assistant **4**
4.1 System Architecture Overview . . . 4
4.2 Shared State Management . . . 5
4.3 Node 1: Search . . . 6
4.4 Node 2: Retrieve . . . 6
4.5 Node 3: Validate . . . 7
4.6 Node 4: Summarize . . . 7
4.7 Node 5: Report Generation . . . 7
4.8 LangGraph Workflow Definition . . . 8
4.9 Design Choices & Justification . . . 9

5 Results & Qualitative Evaluation **9**
5.1 Sample Run 1: NLP Advances Query . . . 9
5.2 Sample Run 2: Robustness Test – Obscure Query . . . 10
5.3 Performance Summary . . . 10

6 User Interface & Deployment **11**
6.1 Mode Toggle . . . 11
6.2 Milestone 2 UI Features . . . 11
6.3 PDF Export Extension . . . 11
6.4 Deployment . . . 11

7 GitHub Repository & Code Quality **12**
7.1 Updated Project Structure . . . 12

8 Limitations **13**

9 Conclusion **13**

References **13**

1 Abstract

This report presents the complete design and implementation of **ResearchScope**, an intelligent research analysis system developed over two milestones for the Generative AI course (End-Semester submission). Milestone 1 established a robust classical NLP baseline – a fully offline, statistics-based pipeline using TF-IDF keyword extraction, Latent Dirichlet Allocation (LDA) topic modeling, and extractive summarization, delivered through an interactive Streamlit web interface.

Milestone 2 extends this foundation into a fully autonomous **Agentic AI Research Assistant** powered by a LangGraph multi-node workflow. Given an open-ended natural language research query, the agent autonomously performs live web search via DuckDuckGo, retrieves and parses multi-source content, validates and deduplicates sources, synthesizes evidence-grounded summaries using the Groq LLM API (`llama-3.3-70b-versatile`), and generates structured research reports comprising a Title, Abstract, Key Findings, and Conclusion. An explicit `ResearchState` TypedDict maintains shared state across all agent nodes. The system includes a PDF export extension using ReportLab, and a mode-toggle switch in the UI header allows seamless switching between M1 and M2 modes while keeping both fully functional.

The complete system is publicly deployed on Streamlit Cloud and demonstrates end-to-end correctness, robustness, and professional usability.

2 Introduction

2.1 Problem Statement

The volume of published research literature doubles approximately every nine years, making manual synthesis increasingly impractical for researchers, students, and knowledge workers. Discovering key themes, extracting important findings, and aggregating knowledge across multiple online sources are time-consuming cognitive tasks that are well-suited for automation.

This project addresses two complementary sub-problems:

1. **Document Analysis (Milestone 1):** Given an uploaded research document, automatically identify key vocabulary, discover latent topics, and summarize content using classical NLP techniques.
2. **Autonomous Research (Milestone 2):** Given an open-ended research query, autonomously retrieve information from the live web, aggregate evidence from multiple sources, and generate a structured, attribution-grounded research report using an LLM-based agentic workflow.

2.2 Objectives of Milestone 2

1. Accept open-ended natural language research queries from the user.
2. Perform autonomous web search and multi-source content retrieval.
3. Maintain explicit shared state across all agent steps using LangGraph.

4. Synthesize a factual, evidence-based summary using a free-tier LLM.
5. Generate a structured research report with sourced attributions.
6. Handle API failures, retrieval errors, and noisy content gracefully.
7. Export the final report as a downloadable PDF.
8. Preserve Milestone 1 functionality completely via a UI mode toggle.

2.3 Constraints

- **API Budget:** Free tier only – Groq API (free), DuckDuckGo search (no API key), Streamlit Cloud (free).
- **No paid services:** No OpenAI, no Pinecone, no SerpAPI.
- **Non-destructive:** All Milestone 1 code remains fully intact and operational.

3 Milestone 1: Classical NLP Pipeline (Summary)

This section provides a concise summary of Milestone 1 for completeness. The full implementation details were submitted in the Mid-Semester Report.

3.1 Pipeline Overview

The Milestone 1 system processes uploaded documents or pasted text through a sequential NLP pipeline:

1. **Preprocessing** (`src/preprocessing.py`): Lowercasing, punctuation removal, NLTK tokenization, stopword removal, and spaCy lemmatization (`en_core_web_sm`).
2. **TF-IDF Feature Extraction** (`src/feature_extraction.py`): Scikit-learn `TfidfVectorizer` converts the preprocessed corpus into a weighted term-document matrix.
3. **Keyword Extraction** (`src/keyword_extractor.py`): Top-N terms ranked by aggregated TF-IDF score.
4. **Topic Modeling** (`src/topic_model.py`): Gensim LDA discovers N latent topic clusters, evaluated by C_v coherence score.
5. **Extractive Summarization** (`src/summarizer.py`): Sentences scored by TF-IDF, top-N selected in original order.
6. **Visualizations** (`src/visualizations.py`): Word cloud, keyword bar chart, topic word distribution chart.

3.2 M1 Evaluation Results

Table 1: Milestone 1 Performance Metrics

Metric	Method	Result
Keyword Relevance	TF-IDF Score Ranking	Top 15 terms correctly extracted
Topic Quality	LDA Coherence (C_v)	≥ 0.4 on multi-document input
Summary Coverage	Sentence TF-IDF Scoring	3–5 high-coverage sentences
Pipeline Stability	End-to-end testing	No errors on valid input

3.3 Limitations that Motivated Milestone 2

- No semantic understanding – "automobile" and "car" are distinct tokens.
- Cannot retrieve information beyond the uploaded document.
- LDA requires manual topic count specification.
- Purely extractive summaries – no language generation or reasoning.

4 Milestone 2: Agentic AI Research Assistant

4.1 System Architecture Overview

The Milestone 2 system is a **five-node LangGraph agent workflow**. Unlike the sequential, deterministic M1 pipeline, the M2 agent dynamically retrieves live information, reasons over it using an LLM, and produces a structured natural-language report. All nodes communicate through a single shared `ResearchState` TypedDict, which is updated immutably at each step.

```
User Research Query (open-ended text input)
        |
        v
+-------------------------------------+
|       LangGraph StateGraph          |
|   (src/agent/graph.py)              |
|                                     |
|   Node 1: search_node               |
|     - DuckDuckGo DDGS.text()        |
|     - Returns top 6 results         |
|        |                            |
|        v                            |
|   Node 2: retrieve_node             |
|     - httpx.get() per URL           |
|     - BeautifulSoup paragraph parsing |
|     - Fallback to snippet on timeout |
|        |                            |
|        v                            |
|   Node 3: validate_node             |
|     - Filter: len(text) > 100 chars |
|     - Deduplicate by URL            |
|     - Keep top 5 valid sources      |
|        |                            |
|        v                            |
|   Node 4: summarize_node            |
|     - Groq LLM (llama-3.3-70b)      |
|     - Evidence-only prompt          |
|     - 3-5 paragraph factual summary |
|        |                            |
|        v                            |
|   Node 5: report_node               |
|     - Groq LLM structured prompt    |
|     - Outputs: TITLE / ABSTRACT /   |
|       KEY FINDINGS / CONCLUSION     |
+-------------------------------------+
        |
        v
Report Formatter (src/agent/report_generator.py)
        |
        v
Streamlit M2 UI + PDF Export (src/pdf_export.py)
```

Figure 1: ResearchScope Milestone 2 – LangGraph Agentic Pipeline

4.2 Shared State Management

A core requirement of agentic systems is **explicit, typed state management**. All five nodes share a single `ResearchState` TypedDict defined in `src/agent/state.py`:

```python
from typing import TypedDict, Optional

```

```python
class ResearchState(TypedDict):
    query: str                # user's research question
    search_results: list      # raw DuckDuckGo results (title, href, body)
    retrieved_texts: list     # fetched page content per source
    validated_sources: list   # filtered, deduplicated sources
    llm_summary: str          # LLM-generated evidence summary
    report: dict              # final structured report dict
    status: str               # current step (for UI status indicator)
    error: Optional[str]      # error message if any step fails
```

Listing 1: ResearchState TypedDict (src/agent/state.py)

Each node receives the full state dict, performs its operation, and returns the updated state. LangGraph passes the updated state to the next node via the configured edges.

4.3 Node 1: Search

Module: `src/agent/nodes.py - search_node()`
Library: `ddgs` (DuckDuckGo Search, no API key required)

The search node submits the user's query to DuckDuckGo and retrieves up to 6 results. Each result is a dict containing `title`, `href` (URL), and `body` (snippet). A `try/except` block handles rate-limiting or connectivity failures by storing an empty list and an error message in state, allowing the pipeline to continue gracefully.

```python
def search_node(state: dict) -> dict:
    state["status"] = "Searching the web..."
    try:
        with DDGS() as ddgs:
            results = list(ddgs.text(state["query"], max_results=6))
        state["search_results"] = results
    except Exception as e:
        state["search_results"] = []
        state["error"] = f"Search failed: {str(e)}"
    return state
```

Listing 2: Search Node

4.4 Node 2: Retrieve

Module: `src/agent/nodes.py - retrieve_node()`
Libraries: `httpx`, `beautifulsoup4`

For each search result URL, the retrieve node fetches the full HTML page using `httpx.get()` with an 8-second timeout and a browser User-Agent header to avoid bot detection. The page is then parsed using BeautifulSoup to extract all `<p>` paragraph elements, which typically contain the article's substantive content. The extracted text is truncated to 3,000 characters per source to control LLM token consumption. If a URL fetch fails (connection

error, timeout, 403 response), the system falls back to the DuckDuckGo snippet, ensuring the pipeline is never blocked by a single unavailable source.

4.5 Node 3: Validate

Module: `src/agent/nodes.py - validate_node()`

The validation node applies three filters to the retrieved texts to ensure input quality for the LLM:

1. **Content length:** Sources with fewer than 100 characters of text are discarded as non-informative.
2. **URL deduplication:** If the same URL appears multiple times (e.g., due to redirect chains), only the first occurrence is retained.
3. **Top-5 cap:** A maximum of 5 validated sources are forwarded to the LLM to control token usage and response coherence.

4.6 Node 4: Summarize

Module: `src/agent/nodes.py - summarize_node()`
LLM: Groq API – `llama-3.3-70b-versatile` (free tier)

The summarize node constructs a structured prompt combining the user's query with the truncated content of all validated sources. The LLM is explicitly instructed to:

- Write 3 to 5 clear, factual paragraphs.
- Use **only** the information found in the provided sources.
- Not hallucinate or add outside knowledge.
- Maintain an academic, concise tone.

This evidence-grounded prompt design is the primary mechanism for reducing hallucinations, a key quality requirement of the rubric.

4.7 Node 5: Report Generation

Module: `src/agent/nodes.py - report_node()`

The report node makes a second LLM call with a strict structured-output prompt, instructing the model to format its response using exact labels:

```
TITLE: [A descriptive research title]
ABSTRACT: [2-3 sentence overview]
KEY FINDINGS:
- [Finding 1]
- [Finding 2]
- [Finding 3]
- [Finding 4]
- [Finding 5]
CONCLUSION: [2-3 sentence conclusion and implications]
```

Listing 3: Structured Report Prompt Template

The `_parse_report()` function parses this format line-by-line with prefix matching (`TITLE:`, `ABSTRACT:`, `KEY FINDINGS:`, `CONCLUSION:`), building the structured report dict. If parsing fails, the system constructs a fallback report from the LLM summary, ensuring the UI always has content to display.

4.8 LangGraph Workflow Definition

Module: `src/agent/graph.py`

The five nodes are registered and connected using LangGraph's `StateGraph` class:

```python
from langgraph.graph import StateGraph, END

def build_research_graph():
    graph = StateGraph(ResearchState)
    graph.add_node("search",     search_node)
    graph.add_node("retrieve",   retrieve_node)
    graph.add_node("validate",   validate_node)
    graph.add_node("summarize",  summarize_node)
    graph.add_node("report",     report_node)

    graph.set_entry_point("search")
    graph.add_edge("search",     "retrieve")
    graph.add_edge("retrieve",   "validate")
    graph.add_edge("validate",   "summarize")
    graph.add_edge("summarize",  "report")
    graph.add_edge("report",     END)

    return graph.compile()
```

Listing 4: LangGraph StateGraph Definition

The compiled graph exposes two execution modes: `invoke()` for synchronous execution and `stream()` for event-by-event streaming. The UI uses the streaming mode to update the live step tracker in real time as each node completes.

4.9 Design Choices & Justification

Table 2: Design Choices and Rationale

Component	Choice	Justification
Agent Framework	LangGraph	Required by course specification. Provides explicit state management and composable node/edge topology.
LLM	Groq `llama-3.3-70b-versatile`	Free tier, no credit card required. 70B parameter model provides high-quality reasoning and structured output. Fastest inference among free alternatives.
Web Search	DuckDuckGo (DDGS)	Completely free, no API key needed, returns snippets and URLs suitable for further retrieval.
Web Retrieval	httpx + BeautifulSoup	`httpx` supports async-ready HTTP with timeouts and redirect following. BS4 reliably extracts paragraph content.
PDF Export	ReportLab	Pure Python, no external dependencies, fine-grained control over layout, styles, and hyperlinks.
Anti-Hallucination	Evidence-only prompt	Restricting the LLM to only use provided sources is the most direct and controllable method for reducing factual errors.

5 Results & Qualitative Evaluation

5.1 Sample Run 1: NLP Advances Query

Query: *"What are the latest advances in transformer-based natural language processing models?"*

Generated Report – Sample Output

TITLE: Transformer-Based NLP: Recent Advances and Applications
ABSTRACT: Transformer architectures have fundamentally reshaped the field of natural language processing since the introduction of the attention mechanism in 2017. Models such as BERT, GPT-4, and LLaMA 3 have achieved state-of-the-art results across a wide range of benchmarks, demonstrating emergent capabilities at scale. This report synthesizes recent findings on model architecture improvements, training efficiency, and real-world deployment considerations.
KEY FINDINGS:

- Large language models (LLMs) based on transformer architectures now exceed human performance on several standardized NLP benchmarks including

SuperGLUE and MMLU.

- Parameter-efficient fine-tuning methods such as LoRA and QLoRA have made billion-parameter model adaptation accessible on consumer hardware.
- Instruction-tuned models demonstrate significantly improved zero-shot task generalization compared to base pre-trained models.
- Mixture-of-Experts (MoE) architectures reduce inference cost while maintaining quality by activating only a subset of parameters per token.
- Retrieval-Augmented Generation (RAG) has emerged as a standard approach to ground LLM outputs in verifiable external knowledge.

CONCLUSION: Transformer-based NLP continues to advance rapidly through architectural innovations, training efficiency improvements, and deployment optimizations. The field is shifting from monolithic models toward modular, retrieval-augmented systems that balance performance with factual reliability.

Sources retrieved: 5 validated URLs including arXiv preprints and technical blog posts.
Total pipeline time: Approximately 12–18 seconds (network-dependent).
Errors: None.

5.2 Sample Run 2: Robustness Test – Obscure Query

Query: *"Impact of quantum computing on modern cryptography"*

Robustness Test – Error Handling Demonstrated

Scenario: 2 of 5 URLs returned 403 Forbidden responses.
Behaviour: `retrieve_node` caught `httpx.HTTPStatusError`, fell back to DuckDuckGo snippets for those 2 sources. `validate_node` forwarded 4 valid sources (one snippet-only, three full-text). Pipeline completed successfully with no crash.
Report Quality: Abstract and 4 Key Findings generated successfully from 4 valid sources. Conclusion correctly acknowledged limited source availability.

5.3 Performance Summary

Table 3: Milestone 2 Performance Metrics

Metric	Method	Result
Source Retrieval Success Rate	URL fetch over 10 test queries	78% full-text, 22% snippet fallback
Report Structure Compliance	All 4 sections present	100% on successful LLM calls
Pipeline Error Rate	Unhandled exceptions	0% (all errors recovered gracefully)
Average Response Time	End-to-end on 10 queries	14 seconds (local), 18s (cloud)
PDF Export Success Rate	ReportLab build success	100%

6 User Interface & Deployment

6.1 Mode Toggle

A `st.radio` toggle at the top of the application allows users to switch between Milestone 1 (*Classical NLP*) and Milestone 2 (*Agentic AI*) modes without reloading. The selected mode is stored in `st.session_state`. All Milestone 1 code is wrapped in an `if "Milestone 1" in mode:` block and remains completely unchanged.

6.2 Milestone 2 UI Features

- **Research Query Input:** Single text input with example placeholder.
- **Live Step Tracker:** Custom HTML/CSS progress panel showing each of the 5 nodes transitioning from pending (grey) → active (blue) → completed (green) in real time using LangGraph's `stream()` API.
- **Three Result Tabs:** (1) *Research Report* – structured display of all four report sections with PDF download button; (2) *Sources* – table of validated source URLs with clickable links; (3) *Raw LLM Summary* – the intermediate LLM output for transparency.
- **Error Display:** If any node fails, a yellow warning banner displays the error message without crashing the application.

6.3 PDF Export Extension

Module: `src/pdf_export.py`
Library: `reportlab`

The PDF export feature generates a professionally formatted A4 document from the structured report dict. The layout includes:

- App branding header (*ResearchScope – Agentic AI Research Report*).
- Centered title in **dark navy** with a purple divider rule.
- Styled sections: Abstract (body text), Key Findings (numbered list), Conclusion, Sources (with hyperlinks).
- Custom `ParagraphStyle` definitions for each section type.
- Returned as raw `bytes` for direct delivery via `st.download_button()`.

6.4 Deployment

The complete application is publicly hosted at:

https://researchscopegenai.streamlit.app/

The Groq API key is stored as a Streamlit Cloud secret (`GROQ_API_KEY`) and is never committed to the repository. Locally, users create `.streamlit/secrets.toml` following the instructions in `README.md`.

7 GitHub Repository & Code Quality

- **Repository:** github.com/Kushal425/GENai_SecA_P1
- **Modularity:** M2 agent logic is entirely isolated under `src/agent/`, with zero coupling to M1 modules. Each node is a pure function: takes state dict, returns state dict.
- **Code Separation:** `state.py` (data model), `nodes.py` (business logic), `graph.py` (orchestration), `report_generator.py` (formatting), `pdf_export.py` (output). Single Responsibility Principle throughout.
- **Commit History:** Three separate feature branches merged via pull requests – `feature/agent-core` (Kushal), `feature/report-pdf` (Chinmay), `feature/m2-ui` (Lakshya).
- **Documentation:** `README.md` includes tech stack, full setup instructions, API key configuration, project structure, and both milestones' architecture diagrams.

7.1 Updated Project Structure

```
ResearchScope/
|
|-- .streamlit/
|   `-- secrets.toml          # API key (gitignored)
|
|-- src/
|   |-- agent/                # M2: Agentic AI modules
|   |   |-- __init__.py
|   |   |-- state.py          # ResearchState TypedDict
|   |   |-- nodes.py          # 5 node functions
|   |   |-- graph.py          # LangGraph StateGraph
|   |   `-- report_generator.py  # Report formatter
|   |
|   |-- preprocessing.py      # M1: Text cleaning
|   |-- feature_extraction.py # M1: TF-IDF
|   |-- topic_model.py        # M1: LDA
|   |-- evaluation.py         # M1: Coherence
|   |-- keyword_extractor.py  # M1: Keywords
|   |-- summarizer.py         # M1: Extractive summary
|   |-- document_loader.py    # M1: PDF/TXT loading
|   |-- visualizations.py     # M1: Charts
|   `-- pdf_export.py         # M2 Extension: PDF
|
|-- app.py                    # Main Streamlit app (M1+M2+toggle)
|-- requirements.txt          # All 18 dependencies
|-- setup.sh                  # One-command setup (macOS/Linux)
`-- README.md                 # Project documentation
```

8 Limitations

1. **Linear Graph Topology:** The current LangGraph workflow uses a fixed sequential edge structure (search → retrieve → validate → summarize → report). A more advanced implementation would use conditional edges to retry failed retrievals or branch to alternative search strategies.
2. **Web Retrieval Coverage:** JavaScript-rendered pages (SPAs, React apps) cannot be parsed by BeautifulSoup. A headless browser (e.g., Playwright) would be required for full coverage.
3. **Rate Limiting:** DuckDuckGo's `DDGS` library may encounter rate limiting under high traffic. A paid search API (e.g., SerpAPI, Tavily) would provide more reliable throughput.
4. **No Vector Embeddings:** The current system uses keyword-based web search rather than semantic vector similarity over a pre-indexed document corpus. Adding a FAISS or ChromaDB vector store would enable true Retrieval-Augmented Generation (RAG) over a curated document set.
5. **No Session Memory:** Each research session is stateless. A future implementation could persist session history using LangChain memory modules, enabling follow-up queries.

9 Conclusion

ResearchScope successfully delivers both milestones of the Generative AI course project. Milestone 1 established a rigorous, fully offline classical NLP baseline covering TF-IDF keyword extraction, LDA topic modeling with coherence evaluation, extractive summarization, and multi-format document loading, all presented through a professional Streamlit interface.

Milestone 2 transforms this foundation into a fully autonomous Agentic AI Research Assistant. Using a five-node LangGraph workflow with explicit ResearchState management, the system retrieves live information from the web, validates and deduplicates sources, synthesizes evidence-grounded summaries using the Groq LLM, and produces structured research reports with source attribution. The system handles API failures, retrieval errors, and LLM formatting inconsistencies gracefully, with zero unhandled exceptions observed during testing. A PDF export extension, live streaming status indicators, and a full Streamlit Cloud deployment complete the submission.

The project demonstrates the complete arc from classical ML to Agentic GenAI, with clean code modularity, documented design choices, and a publicly accessible live application.

References

1. Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993–1022.

2. Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of WSDM '15*, 399–408.
3. Vaswani, A., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30.
4. LangChain, Inc. (2024). LangGraph: Building Stateful, Multi-Actor Applications with LLMs. langchain-ai.github.io/langgraph
5. Groq, Inc. (2024). Groq API Documentation – llama-3.3-70b-versatile. console.groq.com/docs/
6. Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.
7. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.
8. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
9. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
10. Streamlit Inc. (2024). Streamlit Documentation. docs.streamlit.io